24 MANTRA ORGANIC × BLINKIT · REVENUE GROWTH MANAGEMENT

Standard Operating Procedure

Rebuild and operate the discount optimization system — from zero

Written for a new employee or junior analyst with no prior knowledge of this project. Every step states WHAT, WHY, HOW, the EXPECTED OUTPUT, COMMON MISTAKES, and a VALIDATE check. Every phase ends with an exit checklist that gates the next phase.

PHASES

v1.1 · July 2026 · written against repository state git 6fd3f1e+ · companion: Operations Manual (architecture exhibits)

How to use this SOP

This document assumes zero prior knowledge. Each phase contains numbered steps; every step tells you WHAT to do, WHY it exists, HOW to do it (exact commands), the EXPECTED OUTPUT, the COMMON MISTAKES, and a VALIDATE line. Do not skip validation — each phase ends with an exit checklist that must be 100% YES before you continue. Diagrams for everything described here are in OPERATIONS_MANUAL.pdf (keep it open beside this SOP).

Conventions

REPO = the project folder (contains pipeline.py). All commands run from REPO root in a terminal (Windows: PowerShell or CMD). Commands are prefixed python -X utf8 because reports contain ₹ and arrow characters. Paths use Windows style; on Mac/Linux replace backslashes. “Cell” = one SKU in one city — the atomic decision unit (585 today).

Recommended tools & versions (install exactly these or newer)

Tool / library	Version	Used for	Install
Python	3.12.x (≥3.10 OK)	everything	python.org installer; tick “Add to PATH”
pandas	≥ 2.0	all data handling	pip install pandas
numpy	≥ 1.26	numeric core	pip install numpy
scipy	≥ 1.11	DE optimizer, SLSQP	pip install scipy
statsmodels	≥ 0.14	Huber/WLS regressions	pip install statsmodels
scikit-learn	≥ 1.3	Ridge, GBM (DML nuisances)	pip install scikit-learn
openpyxl + xlsxwriter	≥ 3.1 / ≥ 3.1	Excel workbooks	pip install openpyxl xlsxwriter
git	≥ 2.40	version control, recovery	git-scm.com
VS Code (optional)	latest	editing config	code.visualstudio.com
Excel / LibreOffice	any modern	reading workbooks, KAM file	—

One-line install: **pip install pandas numpy scipy statsmodels scikit-learn openpyxl xlsxwriter**. No cloud, no database server, no Gurobi license is required — this system runs entirely on a normal laptop (8 GB RAM minimum, 16 GB comfortable).

WHY NO CLOUD — RIGHT-SIZING

PepsiCo needs Azure + Databricks + a dedicated Gurobi server because they optimize 90+ product groups across 50+ markets with hundreds of thousands of constraints. At 585 cells (and even at 15 brands ≈ 9,000 cells) a laptop solves everything in minutes. Do not add infrastructure the problem size does not demand.

PHASE 0 Environment & tools

Objective. A working Python environment where every library imports and git works.

Connects to. Everything else. If Phase 0 is wrong, every later phase fails with confusing errors instead of clear ones.

STEP 0.1 Install Python and verify

WHAT	Install Python 3.12 from python.org. During install tick “Add python.exe to PATH”.
WHY	Every component is a Python script; PATH lets you run “python” from any folder.
HOW · RUN	python --version
EXPECTED	Python 3.12.x printed. If “not recognized”, PATH was not ticked — re-run installer → Modify → tick PATH.
MISTAKES	Installing from the Microsoft Store (sandboxed, breaks pip in odd ways) · having two Pythons and installing packages into the wrong one (always use “python -m pip …” to be safe).
VALIDATE	python --version prints 3.10+ in a NEW terminal window.

STEP 0.2 Install the libraries

WHAT	Install the seven libraries in the versions table.
WHY	pandas/numpy move data; scipy runs the optimizer; statsmodels fits the regressions; scikit-learn powers DML and shrinkage; openpyxl/xlsxwriter write the Excel deliverables.
HOW · RUN	python -m pip install pandas numpy scipy statsmodels scikit-learn openpyxl xlsxwriter
EXPECTED	“Successfully installed …” lines; no red ERROR lines.
MISTAKES	Corporate proxy blocking pip (ask IT for the proxy flag) · running pip for Python 2 on old machines.
VALIDATE	python -c "import pandas,numpy,scipy,statsmodels,sklearn,openpyxl; print('OK')" prints OK.

STEP 0.3 Install git and set identity

WHAT	Install git; set your name/email; enable long paths on Windows.
WHY	git is the audit trail, the undo button, and (as proven on 6 July) the recovery tool when a sync tool truncates files.
HOW · RUN	git --version git config --global user.name "Your Name" git config --global user.email you@company.com git config --global core.longpaths true
EXPECTED	git version 2.4x printed; config commands return silently.
MISTAKES	Skipping git because “it's just me” — the first time a file corrupts or a change breaks a model, git is the only way back.
VALIDATE	git config user.name prints your name.

PHASE 0 EXIT CHECKLIST — every box must be YES before the next phase

- ☐ python --version shows 3.10+ in a fresh terminal
- ☐ The 6-library import one-liner prints OK
- ☐ git --version works and identity is set
- ☐ You have a terminal you are comfortable opening at any folder (Shift+Right-click → “Open PowerShell window here”)

PHASE 1 Data acquisition & input contracts

Objective. The two raw inputs exist, in the exact expected shape: the platform sales export and the brand SKU master.

Connects to. Phase 3 (the pipeline reads these files). Bad or renamed columns here cause 100% of downstream failures.

STEP 1.1 Create the folder skeleton

WHAT	Create the project root and the input folder.
HOW · RUN	mkdir OptimalPriceFinder cd OptimalPriceFinder mkdir input_data
WHY	input_data/ is the ONLY folder the pipeline scans for raw files; everything else is generated.
EXPECTED	Empty folders exist.
VALIDATE	dir shows input_data.

STEP 1.2 Export the platform sales data (the RCA file)

WHAT	From the Blinkit seller/brand portal export the category-level RCA report — daily grain, ALL brands (not just yours), one file per month for at least the last 6 months. Save into input_data/ named like JAN_2026_BLINKIT_RCA.csv … JUNE_2026_BLINKIT_RCA.csv. Once live, add WEEKLY files every Monday.
WHY	Six months of daily history is the minimum for elasticity estimation (price changes are rare events). ALL brands matters: competitor rows become competitor-pressure features later — do not pre-filter.
EXPECTED	~700–780k rows per monthly file, 29 columns. The 7 REQUIRED columns (exact names): Product ID · City · Date · Offtake (Qty) · MRP · Wt. Discount % · Product Title. Important optional columns the models use when present: Wt. OSA % (availability), Ad SOV, Category, Selling Price, Grammage, Brand, Est. Category Share (MRP).
MISTAKES	Renaming columns in Excel (the loader matches exact names) · opening the CSV in Excel and re-saving (mangles dates to local format) · exporting only your brand · deleting old months to “clean up” (history is the fuel — never delete).
VALIDATE	Open one file's first rows in a text editor (not Excel): header row contains the 7 required names; dates look ISO-like and consistent; multiple brands visible.

STEP 1.3 Build the SKU master (MY SKU.csv)

WHAT	Create input_data/MY SKU.csv listing ONLY your own products. Columns exactly: Platform, Product ID, Product Title, Grammage, Brand, Item ID. One row per sellable SKU (~89 today).
WHY	This is the allowlist that filters the all-brand RCA down to your brand, and the source of pack size (Grammage) used by the price-per-kg ladder.
EXPECTED	A small CSV; Product IDs match the IDs appearing in the RCA files.
MISTAKES	IDs typed with spaces or as text-formatted numbers that gain “.0” · missing newly launched SKUs (review quarterly) · putting competitor SKUs in it.
VALIDATE	Pick 3 random Product IDs from the master; find each in an RCA file (Ctrl+F). All 3 must appear.

DATA FIRST — PEPSICO'S HARD LESSON

Data quality is the #1 documented implementation cost in PepsiCo's paper (“retailers differ in how they define promotional events… requiring substantial harmonization”). Budget real time for Phase 1; every hour here saves ten downstream.

	PHASE 1 EXIT CHECKLIST — every box must be YES before the next phase
☐	input_data/ contains ≥6 monthly RCA CSVs, consistent naming
☐	Header of each file contains the 7 required columns with exact names
☐	Files contain ALL brands (spot-check a competitor name)
☐	MY SKU.csv exists with the 6 exact columns and current SKU list
☐	3-ID spot check passed (master IDs appear in RCA files)
☐	Nothing was opened-and-resaved in Excel

PHASE 2

Repository, configuration & sanity proof

Objective. The complete codebase in place, configured for the brand, with every self-test passing.

Connects to. Phases 3–6 execute this code. The sanity proof (verify_loop) is your license to trust everything later.

DECISION POINT — clone the existing repo vs rebuild from scratch

Approach	Pros	Cons	Verdict
A · Clone the existing git repository (all modules already written, tested, verified)	Hours not weeks · every module carries built-in smoke tests · verified against live data	You inherit code you didn't write (this SOP + OPERATIONS_MANUAL explain every module)	RECOMMENDED — always
B · Rebuild module-by-module from the blueprint (PEPSICO_PRICING_REVERSE_ENGINEERING_BLUEPRINT.md + DESIGN_REVIEW)	Deepest possible understanding · clean-room IP	3–6 weeks of skilled work · high regression risk · you must re-create every validation gate	Only for training exercises or IP isolation

STEP 2.1

Get the code (Approach A)

WHAT	Clone the repository into your project root (or copy the folder, then run git status to confirm it is clean).
HOW · RUN	git clone <your-repo-url> . # or copy the existing folder git status git log --oneline -5
EXPECTED	Working tree clean (or only known doc edits). Recent commits visible (e.g. 6fd3f1e agreement wiring, 82ea05a punchlist fixes).
MISTAKES	Working from a cloud-sync “Copy” folder: sync tools have truncated code files mid-write here before. Keep the WORKING copy outside auto-sync; let git be the backup.
VALIDATE	git diff --stat is empty; no file shows hundreds of deleted lines.

What must exist after this step (top-level inventory)

Path	What it is
pipeline.py + v4_config.py	8-stage data pipeline + ALL thresholds/config in one file
stage1_ingestion ... stage8_output/	pipeline stage modules (ingest → prepare → features → elasticity → curves → economics → guardrails → report)
scripts/analysis/	Engine 1: discount_plan.py · dml_estimate.py · validate_plan.py · build_report.py (+ optimize_plan, unlock_estimate)
scripts/pricing/	Engine 2: pricing_panel.py · elasticity_bayes.py (fallback elasticity_hier.py) · de_optimizer.py · whatif.py · pricing_engine.py
scripts/tracker/	Weekly loop: weekly_tracker.py · actuals.py · killswitch.py · guardrail.py · scorecard.py · seasonality.py · workbook.py · verify_loop.py
scripts/analysis/competitor_features.py	competitor aggregates miner (feeds the planned R5 challenger pass)
input_data/ · v4_outputs/ · DISCOUNT_PLAN/	inputs · run-stamped pipeline outputs · weekly business deliverables + tracker state
doc/ + *.md + OPERATIONS_MANUAL.pdf	architecture and decision documentation

STEP 2.2 Configure the brand (v4_config.py)

WHAT	Open v4_config.py and set/verify the keys that define YOUR brand and safety rails.
HOW	Edit these (everything else keep defaults): BRAND_NAME + OWN_BRAND_PATTERNS (word-boundary brand match); CATEGORY_MODE='column' (uses the platform's Category column); STRATEGIC_SKUS=[...] hero product IDs that must never be auto-cut (get the list from the brand owner); FESTIVAL_DATES / PLATFORM_EVENT_WINDOWS for the next two quarters. Leave the tuned thresholds alone: OSA_OOS_THRESHOLD=50, OUTLIER_Z=2.0, TRAIN_LOOKBACK_DAYS=180, MARGINAL_ROI_THRESHOLD=1.0, HISTORICAL_FLOOR p25/90d, MIN_DISCOUNT_CHANGE_PPT=3, TARGET_TIMELINE_WEEKS=12.
WHY	Config-not-code is the PepsiCo scalability principle: a new brand/market is onboarded by editing config only.
MISTAKES	Over-broad brand pattern (e.g. “Gold”) matching competitor titles — the loader has an over-match guard that raises loudly; fix the pattern, don't disable the guard · leaving STRATEGIC_SKUS empty then being surprised a hero SKU appears in a cut list.
VALIDATE	python -c "import v4_config; print(v4_config.BRAND_NAME, len(v4_config.STRATEGIC_SKUS))"

STEP 2.3 Run every self-test (the sanity proof)

WHAT	Each core module carries a built-in synthetic smoke test; run all, then the end-to-end loop proof.
HOW · RUN	python -X utf8 scripts/tracker/killswitch.py python -X utf8 scripts/tracker/actuals.py python -X utf8 scripts/pricing/de_optimizer.py python -X utf8 scripts/pricing/pricing_panel.py python -X utf8 scripts/tracker/verify_loop.py
EXPECTED	Each prints “All smoke-test assertions passed”. verify_loop prints “LOOP CLOSED: YES ✓” — it simulates a full week: predictions → execution log → actuals backfill → kill-switch → scorecard, on real historical data.
MISTAKES	Running verify_loop AFTER go-live (it RESETS tracker_history/baselines/execution_log — only run it now, pre-live, or on a copy) · ignoring one failing test (“probably fine”) — a failing smoke test is a broken file.
VALIDATE	All five commands exit cleanly. If any SyntaxError appears: git restore (sync truncation) and re-run.

PHASE 2 EXIT CHECKLIST — every box must be YES before the next phase

- ☐ Repo present, git clean, recent commits visible
- ☐ v4_config.py: brand patterns, category mode, STRATEGIC_SKUS, festival windows set
- ☐ All four module smoke tests pass
- ☐ verify_loop prints LOOP CLOSED: YES
- ☐ tracker_history.csv / baselines.json / execution_log.csv are RESET (empty state) after verify_loop — you are pre-live

PHASE 3 Run the data pipeline (raw exports → fact table)

Objective. One command turns raw all-brand CSVs into the cleaned per-cell daily fact table every model consumes.

Connects to. Phase 4 (both engines read v4_outputs//fact_table.csv). Re-run weekly forever (Phase 5, Step 5.2).

STEP 3.1 Execute the pipeline

WHAT	Run the 8-stage pipeline.
HOW · RUN	python -X utf8 pipeline.py
WHY (stages)	1 ingest: chunked read, own-brand filter with over/under-match guards, column validation · 2 prepare: stable_mrp (90th-pct MRP), selling_price, OOS flag (<50% availability), festival flags, z>2 outlier removal, is_regular_day, cell_id · 3 features: logs, lags, rolling means, badge residual · 4 per-category robust elasticity + confidence tiers · 5 discount-response curves · 6 marginal-ROI ladder + elbow · 7 guardrails + action tiers · 8 waste/reinvest Excel + leakage decomposition.
EXPECTED	10–20 min. Console shows per-stage progress; a new folder v4_outputs/<YYYYMMDD_HHMMSS>/ appears containing fact_table.csv (the handoff artifact), features.csv, elasticity_estimates.csv, recommendations.csv, WASTE_REINVEST_REPORT.xlsx/.md, outliers_removed.csv, per_cell_detail.json.
MISTAKES	Killing the run because stage 1 “hangs” (it is chunk-reading ~200 MB files) · running with Excel holding a lock on an output file · panicking at soft warnings (category coverage warnings are informational).
VALIDATE	See 3.2.

STEP 3.2 Validate the fact table (the numbers that must be true)

WHAT	Sanity-check the new run before anything consumes it.
HOW · RUN	python -c "import pandas as pd, glob; r=sorted(glob.glob('v4_outputs/2026*'))[-1]; f=pd.read_csv(r+'/fact_table.csv'); print(r); print('rows', len(f)); print('cells', f['cell_id'].nunique()); print('date span', f['DATE'].min(), '→', f['DATE'].max())"
EXPECTED	Cells ≈ 585 (84 SKUs × 11 cities; yours may differ slightly with catalogue churn) · date span covers all input months · rows in the hundreds of thousands · discount values 0–60%, no negatives.
MISTAKES	Accepting a run whose date span silently lost a month (a malformed file was skipped) — check the span every single time.
VALIDATE	All three numbers plausible; if cells collapsed (e.g. 40), your brand patterns over-filtered — fix config, re-run.

IMMUTABLE INTERMEDIATES

Never edit fact_table.csv by hand, ever. If something looks wrong, fix the INPUT or the CONFIG and re-run the pipeline. Hand-edited intermediates are un-reproducible and silently poison every model downstream — the cardinal sin of data engineering.

	PHASE 3 EXIT CHECKLIST — every box must be YES before the next phase
☐	pipeline.py completed without hard errors
☐	New v4_outputs// folder with fact_table.csv present
☐	Cell count ≈ expected (585); date span covers all input files
☐	WASTE_REINVEST_REPORT.xlsx opens in Excel
☐	You did NOT hand-edit any generated file

PHASE 4 Run the two decision engines

Objective. Engine 1 (causal, per-cell) and Engine 2 (portfolio, cross-price) each produce recommendations; their agreement file is the only path to an executed cut.

Connects to. Phase 5 (the tracker consumes all_cells.csv + agreement.csv every Monday). Re-run this phase every 4 weeks (Phase 6).

STEP 4.1 Engine 1 — causal analysis chain (run in THIS order)

WHAT	Four scripts, each feeding the next.
HOW · RUN	python -X utf8 scripts/analysis/discount_plan.py python -X utf8 scripts/analysis/dml_estimate.py python -X utf8 scripts/analysis/validate_plan.py python -X utf8 scripts/analysis/build_report.py
WHY	discount_plan: weekly panel → per-category confounder-controlled regression (stock, ads, competition, momentum, month) → CI break-even test → buckets (a_stock / b_competitive / c_waste_cut / e_reinvest / f_monitor) · dml_estimate: Double ML strips nonlinear confounding and must re-confirm every cut category · validate_plan: gates C1–C8 (C8 = DML confirmation) · build_report: writes the human documents.
EXPECTED	v4_outputs/<run>/plan/ gains all_cells.csv (585 rows), cut_list.csv, reinvest_list.csv, test_unlock_list.csv, dml_results.json, plan_summary.json · DISCOUNT_PLAN/ gains PLAN.md, DATA_GAPS.md, MEASUREMENT_SPEC.md. Reference values from the live 5-July run: 63 waste-cut cells, ₹6.98L/mo high-confidence, 10/10 cut categories DML-confirmed.
MISTAKES	Running dml before discount_plan (no cut list to confirm) · treating “Experimental” savings as bankable (only High-confidence is banked; the rest routes to tests).
VALIDATE	validate_plan prints ALL gates PASS. Open plan_summary.json: bucket_counts present; meets_target true/false is honest.

STEP 4.2 Engine 2 — portfolio pricing engine

WHAT	One orchestrator runs panel → Bayesian elasticities → DE optimizer → what-if → agreement.
HOW · RUN	python -X utf8 scripts/pricing/pricing_engine.py
WHY	Adds the physics Engine 1 cannot see: cross-price cannibalization between sibling SKUs. Also runs the honesty check — pushing Engine 1's cut list through the cross-price model to confirm cuts hold at PORTFOLIO level (reference: +1.67%, cuts hold). Produces agreement.csv: per cell, does Engine 2 endorse the cut? (reference: 51 of 63 agreed; 12 held “test first”).
EXPECTED	DISCOUNT_PLAN/pricing/: elasticities.csv (with posterior SD — wide band = act only via test), cross_price.csv, pricing_reco.csv, gates.json, PRICING_PLAN.md, agreement.csv, history/<run>/ archive.
MISTAKES	Reading raw own_elast values as gospel — LOOK at own_sd; the report deliberately labels wide-band categories low-confidence · deleting history/ stamps (they are your audit trail).
VALIDATE	gates.json → all key checks pass (reference: pooled R^2 0.892 · wMAPE 0.255 · bias 0.066); agreement.csv has one row per cell.

QUADRUPLE LOCK

Two independent engines with an agreement gate is the same philosophy as PepsiCo's validation-gate stack: no single model is ever trusted alone. A cut needs FOUR locks — waste bucket (CI test) + DML confirmation + Engine-2 agreement + not-a-strategic-SKU — before the tracker will glide it.

PHASE 4 EXIT CHECKLIST — every box must be YES before the next phase

- ☐ Engine-1 chain ran in order; validate_plan gates ALL PASS

	PHASE 4 EXIT CHECKLIST — every box must be YES before the next phase
☐	plan/ contains all_cells.csv + dml_results.json + plan_summary.json
☐	Engine 2 ran; gates.json checks pass; PRICING_PLAN.md readable
☐	agreement.csv exists (one row per cell) and cut-agreement stats printed
☐	Cannibalization verdict says cuts hold at portfolio level (else STOP: investigate before Phase 5)

PHASE 5 Go live — the weekly operating cycle

Objective. The Monday-to-Friday ritual that executes recommendations safely and captures what actually happened.

Connects to. Phase 6 feeds on the actuals this phase produces. This phase repeats every week, forever.

STEP 5.1 MONDAY · drop last week's export

WHAT	Export last week (Mon–Sun) from the portal; save to input_data/ (e.g. W28_2026_BLINKIT_RCA.csv). Same 29 columns.
WHY	Actuals must arrive within days of an action or the kill-switch reacts a month late.
MISTAKES	Skipping a week (creates permanent holes in the register) · partial-week exports.
VALIDATE	File present; covers exactly last week.

STEP 5.2 MONDAY · refresh the fact table

HOW · RUN	python -X utf8 pipeline.py
WHY	The tracker's actuals-backfill needs last week's cleaned rows. Decision models are NOT refit weekly — execution is weekly, learning is 4-weekly (PepsiCo's cadence split; weekly refits chase noise).
EXPECTED	New v4_outputs/<ts>/ with fact_table extended one week.
VALIDATE	Date span now includes last week.

STEP 5.3 MONDAY · run the tracker with actuals

HOW · RUN	python -X utf8 scripts/tracker/weekly_tracker.py --actuals v4_outputs/<NEW_TS>/fact_table.csv --budget_pct 0.125
WHY	In one run: freezes baselines (first time only, persisted) → backfills actuals for open prior weeks → kill-switch judges ONLY applied cut/reinvest cells (confounders excused; 2 strikes → auto-revert + 4-wk freeze; portfolio drift-brake) → agreement gate → festival exclusions → glide ≤3ppt + budget cap → logs this week's predictions (auto label W2, W3…) → scores KAM-confirmed cells → writes outputs.
EXPECTED	Console: “actuals filled: N”, any REVERTS, budget status GREEN/AMBER/RED, cut/hold counts. Files: WEEKLY_READOUT.md, WEEKLY_TRACKER.xlsx, execution_log_template.csv.
MISTAKES	Omitting --budget_pct forever (default cap = whatever you already spend — set the REAL business number) · running twice with hand-typed --week labels (auto-derivation exists; duplicates are silently skipped by design).
VALIDATE	Readout regenerated with this week's date; scored-cells count > 0 from week 2 onward (0 in week 1 is normal).

STEP 5.4 MONDAY · read the readout in this exact order

WHAT	Open DISCOUNT_PLAN/WEEKLY_READOUT.md. Read: 1) ⚠ REVERT/ALERT block — cells to put BACK + drift-brake status (this outranks everything; a revert is the system admitting a mistake before it compounds) · 2) budget line · 3) this week's cuts (each a ≤3-ppt glide step with ₹/wk) · 4) track-record line.
WHY	15 minutes of human judgment is the approval gate — nothing auto-publishes (PepsiCo's human-in-the-loop: recommendations are decision support, not autopilot).
MISTAKES	Cherry-picking extra cells not in the plan · skipping reverts because “let's give it one more week” (the 2-strike rule already gave it one more week).
VALIDATE	You can say out loud, for every cut you approve: current %, new %, projected ₹/wk.

STEP 5.5 TUE–THU · KAM applies on the portal

WHAT	Send the KAM the Weekly Plan sheet (WEEKLY_TRACKER.xlsx) + execution_log_template.csv. They change ONLY listed cells by exactly the listed step, then fill applied = Y/N (+date, reason if skipped).
WHY	The execution log is what separates “model was wrong” from “change was never made”. Without it the scorecard refuses to grade (honest by design).
MISTAKES	KAM applying different depths than listed · KAM returning the file monthly instead of weekly.
VALIDATE	—

STEP 5.6 FRIDAY · file the confirmation

WHAT	Save the KAM's returned file as DISCOUNT_PLAN/execution_log.csv (drop “_template”).
EXPECTED	Columns: week, cell_id, applied (Y/N). Next Monday's run consumes it automatically.
MISTAKES	Leaving the filename as …_template.csv (the tracker will not read it).
VALIDATE	File exists at the exact path with Y/N values.

PHASE 5 EXIT CHECKLIST — every box must be YES before the next phase

- ☐ Weekly export landed Monday and pipeline re-ran
- ☐ Tracker ran with --actuals and a REAL --budget_pct
- ☐ Readout read in order; reverts (if any) actioned FIRST
- ☐ KAM received plan + template; returned Y/N by Friday; file renamed correctly
- ☐ From week 2: “actuals filled” > 0 and scorecard shows hit-rate / realized ₹
- ☐ Calendar holds a recurring Monday 60-min block and a Friday 10-min block

PHASE 6 The learning cycle — 4-weekly refresh & geo-tests

Objective. Models re-earn their job on new data every 4 weeks; contested or big-money claims get controlled city-split tests.

Connects to. Feeds better plans back into Phase 5. This is what makes the system improve instead of merely repeat.

STEP 6.1 Every 4 weeks · full model refresh (champion vs challenger)

HOW · RUN	python -X utf8 pipeline.py python -X utf8 scripts/analysis/discount_plan.py python -X utf8 scripts/analysis/dml_estimate.py python -X utf8 scripts/analysis/validate_plan.py python -X utf8 scripts/analysis/build_report.py python -X utf8 scripts/pricing/pricing_engine.py python -X utf8 scripts/diagnostics/proof_loop.py git add -A && git commit -m "refresh: <date>"
WHY	Four weeks adds ~2,300 cell-weeks of signal — enough to matter without chasing noise.
ACCEPT RULE	Adopt the new models ONLY if: (a) validate_plan gates ALL pass; (b) proof_loop out-of-time accuracy is no worse than the previous refresh; (c) elasticity gates hold (own-price in (−2.5,0), wMAPE<0.40, |bias|<10%). If any fail → keep operating on the previous run (delete/rename the failed run folder so the tracker doesn't pick it up) and investigate.
ALSO	Review kill-switch model-miss cells — the literal list of “where assumptions failed”; they get priority in the refit review.
VALIDATE	A committed refresh with gates recorded; readout next Monday reflects the new plan.

STEP 6.2 Geo-split tests · design and register

WHAT	For cells the system routes to “test first” (engines-disagree; wide posterior SD; big-money claims like Dal = 58% of savings; reinvest pilots): run a controlled city-split test.
HOW	One category per test, ONE change (e.g. Tur Dal 22% → 19%). Rank its cities by volume + trend; pair similar cities; COIN-FLIP each pair into test/control (≥2 test + ≥2 control). Write the pass/fail rule BEFORE starting (e.g. “DiD ≥ 0 → roll out; < −2% → revert + tag model-miss”). Register the test as a row in DISCOUNT_PLAN/TEST_REGISTER.csv: test_id, hypothesis, category, test_cities, control_cities, start, end, rule, status, result. Run 3 weeks. Cap: 3–4 concurrent tests, one per category, ≤10% of weekly discount spend.
WHY	History cannot prove causality (you never see the without-world); a control group makes rain, festivals and app changes cancel out. This is the top rung of the evidence ladder.
MISTAKES	Choosing test cities by gut (use the coin) · changing ads mid-test · peeking at week 1 and stopping early.
VALIDATE	Register row complete BEFORE the first price change.

STEP 6.3 Geo-split tests · measure and decide (difference-in-differences)

WHAT	After 3 weeks compute: effect = (test after − test before) − (control after − control before), using tracker_history actuals (before = 4-wk pre-test mean; after = 3 test weeks mean).
EXAMPLE	Test cities: 1,000 → 960 units/wk (−4%). Control: 800 → 768 (−4%). DiD = (−4%) − (−4%) = 0% → the cut cost nothing while saving 3 points of discount on every unit → PASS, roll out via normal glide.
VALIDITY	Before believing any result: OSA and Ad-SOV moved <10% on both sides during the window — else extend one week (the week was confounded, not informative).
AFTER	PASS → remaining cities via glide (kill-switch keeps watching) · FAIL → revert, tag model-miss, feed the next 6.1 refresh. Either way: update the register row and let the test weeks become training data.
VALIDATE	Register row closed with result + action; decision matches the pre-written rule.

ANSWERS HAVE EXPIRY DATES

Validated answers expire: elasticity is not a constant of nature — competitors, seasons and inflation move it. Re-test a category only when the model's recommendation changes materially or a drift alert fires; a clean answer holds roughly a quarter (PepsiCo revalidates every planning cycle on the same logic).

	PHASE 6 EXIT CHECKLIST — every box must be YES before the next phase
☐	4-weekly refresh executed in order and committed to git
☐	Champion/challenger accept-rule applied (gates + backtest no-worse)
☐	Model-miss cells reviewed in the refresh
☐	TEST_REGISTER.csv exists; every live test has a pre-written decision rule
☐	DiD computed with validity check; decisions follow the pre-written rules
☐	No more than 3–4 concurrent tests; one per category

PHASE 7

Maintenance, troubleshooting & scaling

Objective. Keep the system healthy quarter after quarter, recover from the known failure modes, and onboard brand #2…#15 by config.

Connects to. Loops back into Phases 1–6 for each new brand or quarter.

STEP 7.1 Quarterly maintenance (30 minutes)

WHAT	1) Update festival windows in scripts/tracker/seasonality.py for the next two quarters (approximate weeks are fine) · 2) revisit --budget_pct with the brand owner · 3) refresh STRATEGIC_SKUS · 4) review DATA_GAPS.md and pick ONE gap to close next quarter · 5) archive old v4_outputs/ (keep last 6 runs + any run referenced by tracker history) · 6) zip DISCOUNT_PLAN/ + latest run as an off-machine backup.
WHY	The festival calendar is the only hardcoded knowledge that silently rots; budgets and hero lists drift with strategy.
VALIDATE	Next two quarters' festivals present; backup zip exists off-machine.

STEP 7.2 Onboard a new brand (the 15-brand path)

WHAT	Per brand: new repo copy (or brand subfolder) → Phase 1 with THAT brand's exports + SKU master → edit v4_config.py (brand patterns, strategic SKUs, festivals) → Phases 3–4 → run scripts/diagnostics/data_readiness_report.py → go/no-go → Phase 5.
WHY	Zero engine-code changes per brand is the design goal (PepsiCo: “a new market is onboarded by creating a configuration file”). The readiness report is the honest gate: it grades data depth, price variation and actionable-cell share GREEN/YELLOW/RED before any promise is made to the brand.
MISTAKES	Promising savings before the readiness report · mixing two brands' files in one input_data/.
VALIDATE	Readiness verdict GREEN/YELLOW documented; brand owner has seen it.

Troubleshooting — the known failure modes

Symptom	Cause	Fix
SyntaxError importing killswitch/tracker	cloud-sync truncated files mid-write (happened 6 Jul)	git status → git restore ; re-run Phase-2 smoke tests
“No all_cells.csv — run discount_plan first”	tracker before Engine 1	run Phase 4.1
Readout: “scored cells: 0” after week 2	execution_log.csv missing / still named _template / applied not Y/N	fix the KAM file (Step 5.6)
A week appears skipped	same week label already logged (idempotent append)	by design; to redo a week delete its rows from tracker_history.csv first
Everything flagged confounded	platform-wide OSA collapse	correct behaviour — fix availability; those weeks don't strike
Gates fail at a refresh	thin/odd new data, category churn	champion stays; inspect per-category coverage in gates.json
Cells collapsed to a handful in Phase 3	over-broad/wrong OWN_BRAND_PATTERNS	fix config; re-run pipeline
DE groups_failed > 0	degenerate 1-SKU category×city groups	safe to ignore if small; they hold at current discount
Engines disagree on many cells	confounded categories (expected)	route to Phase-6 tests — that is the designed behaviour
₹ or → prints as boxes in console	terminal encoding	always run with python -X utf8

System-health KPIs (review monthly, present quarterly to the brand)

Hit rate (predicted direction correct; healthy systems trend toward PepsiCo's ~85% acceptance-quality bar) · decision-model MAPE at 3-ppt bins · revenue bias (systematic over-promise is a red flag even when small) · cumulative REALIZED ₹ vs the model's claim (the register is the final proof) · % recommendations applied (ops health — below ~70% means the KAM loop is broken, not the model) · reverts per month (a few is healthy learning; a spike means retrain).

	PHASE 7 EXIT CHECKLIST — every box must be YES before the next phase
☐	Quarterly items done and dated in a maintenance log
☐	Off-machine backup zip exists (DISCOUNT_PLAN + latest run + git bundle)
☐	Troubleshooting table pinned where the operator can see it
☐	Health KPIs reviewed monthly; quarterly summary sent to the brand
☐	New-brand onboarding used the readiness report BEFORE any savings promise

The whole SOP in five lines

Phase 0–2 (once): environment → data contracts → repo + config → sanity proof (LOOP CLOSED: YES). Phase 3–4 (first run + every 4 weeks): pipeline → Engine 1 (causal) → Engine 2 (portfolio) → agreement. Phase 5 (every week): export → refresh → tracker → readout → approve → KAM applies → Friday Y/N. Phase 6 (continuous): refresh with champion/challenger; geo-tests for contested and big-money claims; decisions by pre-written rules. Phase 7 (quarterly): calendars, budgets, backups, readiness-gated onboarding of the next brand.